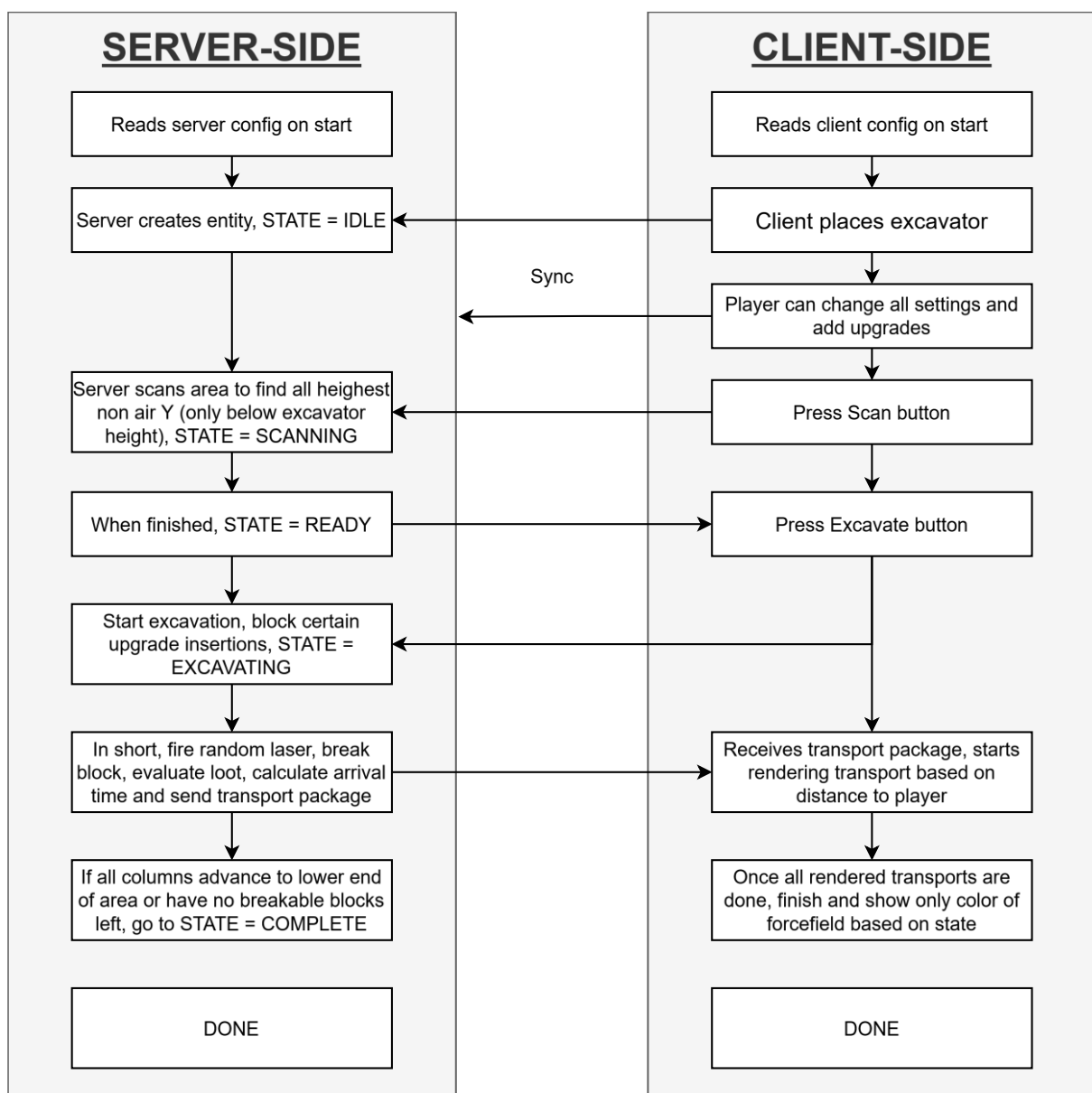


Laser excavator (rough) architecture description

General terms I will use and might not be clear to everyone:

- **Server-Client:** Minecraft always uses a server (also in Singleplayer). The server holds the world state and synchronizes it with the clients (the players). In this mod the server cares about the excavation and the client all rendering
- **Cheap/Expensive:** When I refer to these, I typically mean performance, cheaper is better
- **Cache:** Data that is stored temporarily to improve performance

General functionality:



This is a small (not 100% complete) description of how the general block functions when successfully mining a block. The server block iterates between states (which for example

define the text shown in the UI below the progress bar or the forcefield outer color). IDLE is the normal state, SCANNING is only use during the scan process (orange color), READY is used when the excavator can start mining, EXCAVATING is used when the mining is successfully running (has technical 2 substates, NONE which is normal and LASER when the laser is fired), STORAGE_FULL is used when the mining is stopped when the (well) storage is full. COMPLETE is used only when all columns reach the end.

There are two conditions outside of these general states (which I might merge at some point into the state system). They are OUT OF ENERGY and OVERHEATING. They also effect the text and stop the excavation but don't have separate states as the overheating is only relevant for the nether and affects multiple states and the energy should not fully stop the excavation (so that once energy is again available the machine can directly continue).

General Block Breaking Process:

So, when a block is broken it is removed, drops are sent to the excavator and then added to the inventory, easy and done right? Well for optimization reasons it is a lot more complicated.

1. The server chooses a random column to break a block
2. Calculate loot (using the "learning" loot cache) / apply upgrades like smelting
3. Checks if there is space in the inventory
4. If yes, break block, if no go to STORAGE_FULL state / Direct removal can happen by directly accessing the Chunk data where it is safe (not entities attached etc.) or a block break event
5. Server sends a package to the player notifying the removal and start of a transport (uses batches of changes, refer to ExcavatorBlockSyncBatcher .java, this is cheaper for performance)
6. Client starts transport, updates local (only visual) chunk data only after a short period to update large sections and not every single block (otherwise chunk rendering becomes very slow), time depends on distance of chunk to the player, the closer the faster it updates (only relevant for blocks removed by this mod, all other block updates work as per usual)
7. When starting the transport, the server calculates an arrival time and once it is done it again checks the inventory
8. If full, wait for adding the item, if there is space add it
9. Pending transports (btw in code also called deliveries) periodically check if there is now space (at that point the rest of the excavator has shut down due to full inventory). If you ask yourself "why not reserve space for the item so that it fits", I did this before and it constantly clocked the machine with on the way transports when every new item was immediately removed by a pipe. So this is (I think) the best compromise
10. Once the item is added the transport is done and all data related to it cleared

Column State and Scanning:

The excavation area is decided by the player input and constrained by the settings of the config. Instead of keeping track of every block the excavators keep track of “columns”. An `ExcavatorColumnState` tracks all information regarding the columns of an excavator. During scanning each column starts at the excavator’s height and scans downwards looking for the first non-air block. It does not automatically scan the volume at once but incrementally (budget can be adjusted in the config) to not cause a lag spike on the server.

Importantly, Scanning does not load chunks, so columns in unloaded chunks are also not scanned. The final result is a 2D array of heights used for excavation. That is also the reason why when you place blocks during excavation above the last broken block they will be ignored. When a laser breaks a block the column advances downwards,

For optimization there is also a list of active columns, if a slot is finished it is removed from that list leading to it not being evaluated further for target selection. Also very important, there is an internal “repair” mechanism if the column state points to air. If the block is air (because of a cave for example) and selected, then it advances immediately downwards to the next solid block.

All of this works fine for one excavator, but what happens when two overlap? They constantly mess up each other’s column state leading to a lot of “repairs” which need server performance. Hence if two excavators (without filters) overlap they share a pointer onto the same column. Those are only stored during runtime and per dimension. These pointers store a number of users and are automatically removed if it drops to 1, then the excavator again refers to only its internal state.

Energy System:

Without upgrades an excavator needs 640 FE to break a single block so 32 FE per tick to continuously work. So, speed upgrades also increase power usage as usage per block stays the same. This is only reduced with efficiency upgrades.

When the machine is excavating it looks into its storage whether there is enough FE then removes it and advances one tick. If there is no energy it pauses.

The internal power generator:

To allow the mod to function as a standalone it has an internal power generator. It takes any burnable item and transfers it into FE. For each burn tick (which is the same as burning it in a furnace) it produces 10FE while processing 32 burn ticks per server tick leading to 320 FE/tick in energy. I chose this value as it allows the machine to continuously run with a Speed 5 and Efficiency 5 upgrade. Without efficiency it will not be able to handle the speed of high tiers.

How much can this now generate in blocks? Without efficiency upgrades one coal block allows 250 blocks to be broken, with efficiency 5 it becomes 500. If you (for example for a mod pack) want to only use external energy to nerf the very strong internal generator, it can be fully disabled in the server config.

Removing a block from the world:

This was in the past the most performance heavy server-side work. So, when a laser destroys a block it first checks again whether it is valid. Keep in mind that only the server has the authority to decide what blocks exist, even if a client desyncs that does not matter for a block removal. It is also only removed if its whole loot can fit at that moment into the excavator (if it can't the excavator is set to STORAGE_FULL).

Now there are two ways to remove a block

- A) Direct removal from the chunk data (can be disabled in the config) and is only active if neighbor updates are suppressed and the blockstate allows it (so no entity attached, no redstone wire block, has a block ticker, or interacts in any other way with redstone). Which blocks are allowed is again cached for performance. All of this is done as directly changing the chunk data is faster than using a setBlock() command.
- B) The normal path, invokes the setBlock() command (in Java obviously). Is used if A) is disabled or not possible. This allows for neighbor updates (if enabled, costs performance and can make fluids funky).

This is the server side which is always up-to-date to not allow an incorrect world state that all players see. However, the client updates are for now suppressed. Once a block is removed on the server it calls the ExcavatorBlockSyncBatcher. Here its position is transformed into a 12bit representation. As each 16x16x16 section has only 4096 possible positions (which is exactly 2^{12} or 12bit) this can perfectly describe all positions in one area and is smaller than X, Y and Z in integers. Now the batcher collects updates for each section. The time is depending on the distance to the player (from 5 ticks below 48 blocks to 60 ticks further away than 160 blocks).

However, if a lot of changes happen in one section it can send its package earlier, if it has at least 8 blocks accumulated, the batch is at least 5 ticks old and there have not been a new block in 3 ticks. Once the package is ready to be sent it checks for "listening" players. If there are none it is discarded (this means no player has the chunk loaded so it is not needed as loading the chunk means for the player to also load the new version (as the server version is already up-to-date)).

When the client receives the package it internally updates the chunk, however it does not yet send a command to rebuild the chunk mesh which is used to render. (If other block changes outside this mod change the chunk mesh it will be instantly rebuilt, I only suppress my changes). If now all blocks are locally removed the package sets the chunk dirty leading to it being rebuilt with all changes at once (and not each change). This might lead to the terrain looking like it updates slower especially in great distance but otherwise it did not load at all, as the game was so busy constantly rebuilding meshes, close by chunks will still look correct. You will mostly see this effect in extremely large arrays of excavators.

Fluid removal works the same however with neighbor updates (or ice freezing) it can behave weird. When for example water is updated it might flow into a block above the

current column state leading to the excavator missing it. However I am for now fine with that as otherwise I would need to check the whole area constantly (which is expensive) and there might be certain cases where an excavator could go into an infinite loop when encountering infinite water sources.

Inventory Check and Insertion:

A system I am quite proud of :).

When profiling the performance, the frequent check with the inventory lead to some avoidable server time. So instead of checking with the real inventory to find empty slots I use a (in my opinion) much better system.

Every item (and the empty slots) is represented as a 32bit integer represented as 32 bits. Whereas every slot is defined as a bit. So, assume we have an 8-slot inventory with:

Slot: 7 6 5 4 3 2 1 0

0: Stone, 1: Empty, 2: Dirt, 3: Stone, 4: Empty, 5: Iron, 6: Stone, 7: Empty

The bit values look like this:

Stone -> 01001001, Dirt -> 00000100, Iron -> 00100000, Empty -> 10010010

If we now want to know where the stacks of Stone are we only have to look for the number of trailing zeros which equals the slot where the item is. If we then set this slot to 0 (in the binary) we can again check for the number of trailing zeroes to get the next index. So we do three calculations to get all indices which we could check for space which is much faster than iterating them all and very memory efficient as it only stores one integer per item type. Similarly, we can look for empty slots.

Multi-drops are a bit more complicated. It is not possible to just check for space for both items separately as there might only be one free stack but we need to store two stacks of separate items (I know not so realistic but could theoretically happen). So it copies the inventory, does a test insert, then checks for the next item whether it fits. This is more expensive then the other but correct (and this case is also quite rare)

For insertion we start with the found slots which still have space, insert it and with NeoForge the number of leftover items is returned then we go to the next stack until either all is stored or we need an empty slot. Empty slots work the same but are used last. Obviously, the bitmasks are now updated to reflect the changes. Similarly, if items are removed, the bitmasks update.

If the block is newly loaded or the actual inventory insertion fails, the bitmasks are all dropped and rebuilt to avoid errors. (Though during testing the V1.0, failures never happened).

So how useful is it? Using checks for all slots takes around **600ns** (nano seconds) per item check, now it takes around **250ns**, so around **58%** less. So this sounds tiny, but for thousands to millions of checks it makes a difference.

Loot Calculation:

When a block is broken the server needs to calculate what it drops (taking upgrades into account). Just evaluating the loot table every time is however expensive (can be cheaper).

The general process is as follows:

1. Check whether we have silk touch or fortune upgrade
2. Look whether item is in the loot cache
3. If yes use it, if no evaluate loot table -> add into cache system
4. If auto-smelt, then smelt if possible
5. Then check inventory and continue

In addition, the excavator evaluates loot tables with a theoretical netherite pickaxe.

The loot cache:

The cache is server wide (so all excavators use the same information. It uses a key combined from the fortune level, silk touch yes or no and block. Each cache entry has 3 possible states. LEARNING, CACHED and UNCACHABLE. If an item is first broken it is added to the cache as LEARNING and stores an observation number. If the exact same loot table result is observed 256 (can be changed by CONFIRMATIONS_REQUIRED) times it becomes CACHED and is used instead of direct loot table calls. If the drop differs in any way (for example dropped item, number of items, data components of the item etc.) then it becomes immediately UNCACHABLE and future calls immediately evaluate the real loot table and don't check the cache. Some blocks are immediately rejected such as ores without silk touch or blocks with entities attached. After a set number of cache entries (1024, change with AUDIT_INTERVAL_HITS), the loot table is evaluated once. If the drop is different the cache entry is emptied (it is a leftover of testing, however I keep it for safety)

Another optimization: transports need to know what drops they transport; however, they only have a reference to the cache entry until the item is added to the inventory then this item is added (saves memory).

Now to the performance difference:

A single loot table calculation before needed around **6us** (mikro seconds). It now needs only **0.6us** (so a **90% reduction**).

Delivery/Transport management (on the server-side):

While briefly talked about in other parts of this documentation, this section is exclusively for what happens with an item from the point it is broken to the inventory (only on the server).

Once a block is broken the server calculates an expected arrivalTime based on the path it needs to take. It is then put into a Hashmap where each entry is a list of transports with the same arrival time. Each tick the excavator checks this HashMap whether the tick has an entry. If yes it is processed independent of the excavator state. The excavator also keeps track of the last server tick it has processed something. If it now gets unloaded (so

does nothing anymore) it needs to continue later. If it detected that it is behind that counter schedules a catchup processing all older transports at once. With that, transports can't get lost or stale.

Each transport contains a reference to a cache where all items in transport are cached. This keeps RAM usage low as transports are just cheap references to the actual item, even if thousands of transports contain the same item. Once it reaches the block the item is then copied and transferred to the inventory.

When the arrival time is reached it checks if there is space in the inventory, if yes, it is inserted, if not it reschedules the transport and tries again after a set time (configurable). All transports are saved on the server, so while the connected rendered object is not persistent between restarts, the actual item is persistent and not deleted until successfully put into the inventory or the machine being removed.

Rendering of force fields/lasers and transports:

The rendering is the most expansive client-side part. To allow the rendering of even hundreds of excavators I implemented tons of optimizations. This tries to summarize all of them.

The excavator area subregions:

So, each excavator's area is split into 16 x 16 x 16 subareas. These are used for example for the culling to only check the group and apply the same effect to the whole area instead of calculating this for singular transports. Once a transport enters an area it already calculates its expected exit (and hence when recalculation is needed). This is used to not continuously check if the transport is still in the area but only check it when it is close to an expected border, saving tons of calculations.

General optimizations:

1. All item cubes render only 3 faces (as only 3 are needed), based on the view the sides needed are calculated
2. If the whole excavation area is out of view the whole excavator is culled and nothing rendered and most calculations reduced
3. Items are only rendered as 2D textures facing the camera (which is much cheaper than the item model)
4. All texture information is cached
5. Groups of transports are similarly culled if outside of view (see later)
6. Tons of calculations are done only once and stored (that is the reason why there are tons of variables in the classes)
7. A lot of data is batched and all rendered at once (to increase throughput to the GPU)
8. Light levels are only calculated for whole areas and used for all transports
9. LOD tiers for Force Fields/Lasers and Transports
10. A cap (that can be changed in the config) on how many lasers or transports can be rendered per excavator, if it is reached new transports are not rendered but exist on the server, so the item is still correctly mined)

Level of detail:

Depending on the distance to the player the objects rendered become less detailed or are hidden to save performance without strong visual effect.

The Force Field:

It has three levels, FULL, MID and FAR, all are based on distance which can be adjusted in the config. By default, they are 80 and 160 blocks. The outer rim of and the pillars are always rendered. The inner grid goes from all 4 blocks to 8 to 16. In distance they also become a bit darker to be less visible. The FULL pillars are 8 lines and a small cross at the bottom and a node on top. The sweeping line is also 3 lines.

In MID the pillars center becomes 1 line the bottom node is removed and the beam to the block is reduced to one line.

For FAR pillars, they are collapsed to two parallel lines.

Also, a general optimization here, all values that are the same across all excavators are shared between them and not calculated for each.

The Lasers:

Here we again have the three levels, FULL, MID and FAR. However, as there are much more lasers the default thresholds are lower (at 48 and 92). A full laser a bright line in the center and 4 outer pulsating lines. In the middle distance this pulse is disabled and the laser are only 3 lines looked at the camera faking a 3D effect. The FAR tier only renders one bright line as the center.

Furthermore, the MID and FAR tier skip lasers (so not all are rendered). MID skips around 33.3% of laser and FAR around 50%. This does not really affect the view as on distance most lasers already overlap.

For FAR lasers there is also a shortcut for calculating the distance. Typically, we would check the distance for every laser, but for very far lasers if the whole area (and the entity) is further away than 92 blocks we can immediately classify all lasers as FAR and skip direct calculation.

As before calculations are shared between excavators and there is a (config set-able) cap on lasers per excavator.

The Transports:

As transports are by far the most rendered thing they are also the most expensive one in total. So, it requires more complicated optimizations.

First, we again split them into three tiers: INDIVIDUAL, BATCHED and HIDDEN with thresholds of 100 and 156. Keep in mind all transports in one area are rendered the same to reduce distance calculations. But before a much simpler optimization.

For a transport to know its path it only needs four points which can all be derived from the start point. First the start point sent by the server, then the point where it meets the force

field, then the point on top of the excavator and then the excavator. So, these in combination with the time and speed set in the config is all the block needs to calculate its position for every render call, so it does not need to ask the server (the server does not even know that) or do some other crazy calculation. And the best, for lift and descend it is just one addition of a constant. With the diagonal being a little bit more complicated.

Now lets talk about the tiers:

Individual (up to 100 blocks away):

Here, every transport is rendered, however, items and blocks are handled a bit differently.

First blocks:

- Until 20 Blocks away transports are rendered as 3 faces of a textured cube. They receive full lighting and shadows.
- Then from 20 to 40 they are only a 2D texture with the block's texture. These also receive full lighting. With distance they also reduce in size to lower GPU usage.
- At 40 until 64 they are rendered as "large" dots colored in the average texture color (which again is cached and reused), lighting is only periodically updated as factor to the color (to make it lighter or darker)
- From 64 to 100 they are only rendered as small dots which is extremely efficient

Items:

- Items start as a 2D texture looking at the camera, as items are rarer they stay like this until 64 blocks away
- Then until 100 they are only small dots as before

Batched (100 to 156 blocks away):

At this distance the dots are far away and mostly out of direct view. Hence, I think that not all must be rendered. So at 100 blocks a maximum of 192 transports per area are rendered. It is then reduced to 8 transports at 156 "exponentially (falls of faster at distance). It uses quantization to avoid fast changes at small camera changes. In addition, transports that are rendered are stable between frames (so that they don't flicker)

Hidden (after 156 blocks):

The simplest tier, all transports beyond 156 blocks are not rendered at all.

Others:

- All dots are batched between excavators and then sent to the GPU at once
- Transports that are removed, spread the work across frames to not lead to potential large spikes in CPU usage
- Similar to before, there are caps on maximum transports per excavator

The Upgrade System:

Each excavator can be upgraded by up to seven (number can be adjusted by the config) upgrades. Again, the installed upgrades are not constantly checked but cached when changed and then reused much cheaper. The following table shows all currently implemented upgrades and whether they can be changed while the excavator is running (so it is live updating).

Upgrade	Tiers	Primary subsystem	Live while running?
Speed	I-V	Excavation	Yes
Energy Efficiency	I-V	Energy usage	Yes
Luck	I-III	Loot	Yes
Fluid Ignore	I	Target/column	Yes, with special transition
Silk Touch	I	Loot	Yes
Auto-Smelt	I	Loot	Yes
Nether Cooling	I	For the nether	Yes
Area	I-III	Scanning	Requires area/scan handling
Filter	I-V	Target/column	Restricted compared with simple live upgrades

All upgrades can be configured; be it values or crafting recipes.

Speed upgrades:

Typically, the excavator takes 20 ticks per block removed. This is by default around halved per tier. So, it goes from 20 to 10 to 5 to 3 to 2 to 1 tick. Energy usage obviously also increases.

Energy upgrades:

Each energy upgrade reduces the energy needed per excavation. It is reduced by 10% to 50% linearly.

Luck:

Applies Fortune 1 to 3 when the excavator breaks a block.

Fluid Ignore:

Well, it ignores all fluids.

Silk touch:

Applies silk touch when the block is broken.

Auto Smelt:

Evaluates one furnace melting for each broken block.

Nether cooling:

The excavators are overheating in the nether, to allow them to work in the nether this upgrade is needed.

Area:

Allows an upgrade from 16x16 to 32x32 to 64x64 to 128x128.

Filter:

The filter upgrade allows access to a filter menu, where blocks can be added that are ignored during the target selection. The tier increases the number of filters that can be set from 1 to 4 to 8 to 12 to 16. Additionally, when evaluating a block that should be ignored the machine still goes on cooldown as if it was broken (Tier 1). Further tiers reduce that cooldown by 25 percent each until the Tier 5 where targets are instantly skipped and the next one immediately tested until a breakable block is found (then it goes on cooldown).

Networking:

Again, mentioned all around, this section discusses the networking and how server and client synchronize. As before the server always hold the true world state which is communicated to the client whenever it changes, however especially for the rendering the clients have more freedom to keep traffic low.

1. The menu. This is very simple, each button uses Minecraft vanilla behaviour where the game observer the clients and a package is send to the sever once a button is pressed
2. Blocks removed, this is already discussed in depth in the block breaking section, but in short the server batches block removals in each 16 x 16 x 16 area of the excavation area and send them after a few ticks to the client, where the client then changes its data. Only players that have said area loaded get this package.
3. Visual Events. Transports (and lasers) are also batched on the server, first per dimension than per area/section. If multiple excavators share a section they also batch there transports together.

Batched transports are sent after either a set time or number of batched transports, so neither can there be large bursts by one package nor a large delay.

The packages are then also optimized. All values are stored in arrays which contain all start coordinates for example. Instead of each transport saving its loot on its own, the package contains a palette which encodes each item as a number, which is then used as a transport. So if there are 100 stone transport the item id is send only once.

Automation:

The excavator block exposes automation capabilities to allow it to connect to pipes/cables of other mods and vanilla hoppers. All sides allow FE cables to attach and supply it with energy. Item pipes can similarly connect at all sides to take the items as output or supply fuel. Hoppers can similarly connect at the top and sides to input fuel and extract items on the bottom.

Upgrades cannot be supplied automatically.

Config:

The mod contains a server and client config, which is automatically generated with default values which can be changed to customize the experience. The default values are only written to the config if there is no config available.

On the server side, the config allows you to change all balancing values, such as base speed, area, scan speed, energy capacity, upgrade slots etc. There are also more complex ones such as neighbor update suppression or a list of which blocks cannot be broken.

Furthermore, all upgrades can be customized from specific speed or efficiency they give, to crafting recipes, to the possibility of disabling crafting recipes and much more. There are also debug/test commands which are typically disabled which allow the generation of large grids I used for testing.

The client config allows changes to the rendering, from disabling all transports to changing distances of LOD tiers, to debug modes which are more relevant for testing the mod.

Profiler:

The whole mod contains a profiler that tracks a lot of information to understand which part of the mod requires, how much time, where things could go wrong etc. It is typically disabled and not needed for the gameplay. However, it can be enabled with `/excavatorprofiler start`. `/excavatorprofiler` then shows an overview of performances measured after the start until the command. `/excavatorprofiler detailed` gives a more complex overview. `/excavatorprofiler stop` stops the tracking and prints the simple overview.

Closing Words:

If you spend the time reading this, first of all thank you. I had a ton of fun working on all of this and if you enjoy it or find it helpful then I achieved what I wanted. If you find any bugs or have ideas on how to improve (or extend) the mod feel free to send me a message :).

Best FrozenPhoenix!